Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 03

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Diseñar e implementar Models en Django**, definiendo entidades, campos y relaciones básicas de acuerdo con los requisitos de una problemática planteada.
2. **Aplicar Django ORM y SQLite para gestionar datos persistentes**, utilizando migraciones y operaciones de creación, consulta, actualización y eliminación de registros.
3. **Implementar y explicar el flujo CRUD completo en Django**, integrando URLs, Views, Models, Templates y operaciones ORM para resolver una problemática real.

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- *No aplica*

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Procedimiento

En la semana anterior se trabajó la estructura básica de una aplicación Django utilizando el patrón MVT, formularios, Views, URLs y Templates, pero los datos se almacenaban de forma temporal en memoria. En este laboratorio se dará el siguiente paso: convertir esas estructuras en información persistente mediante Django Models, migraciones, Django ORM y SQLite, de modo que los registros se mantengan disponibles incluso después de reiniciar el servidor.

El laboratorio se desarrollará en dos partes. En la primera, los estudiantes retomarán la aplicación realizada en la Semana 2 y adaptarán su funcionamiento para reemplazar los datos estáticos por Models reales y operaciones ORM, implementando la consulta y el registro de información de manera persistente. En la segunda parte, deberán investigar una problemática real, identificar sus requisitos y diseñar una solución que incluya varias entidades, relaciones entre Models y operaciones completas de creación, consulta, actualización y eliminación.

El objetivo es que el estudiante comprenda cómo Django conecta el código Python con una base de datos mediante el ORM y pueda aplicar este flujo de forma integral:

→ **Request:** El usuario realiza una petición desde su navegador (por ejemplo, acceder al listado de pacientes mediante una petición GET o enviar un formulario por POST).

→ **URL:** El archivo urls.py actúa como enrutador, analizando la dirección web solicitada y dirigiendo el tráfico hacia la función correspondiente.

→ **View:** El archivo views.py recibe la petición. Aquí reside la lógica central, decidiendo qué información se debe consultar, guardar o eliminar.

→ **Model:** Si la vista necesita datos, recurre a las clases definidas en models.py, las cuales representan la estructura exacta de las tablas de la base de datos.

→ **Manager / QuerySet:** Para interactuar con los datos, la vista utiliza el "Manager" por defecto (objects). Mediante QuerySets (ej. objects.all() u objects.filter()), se definen las consultas en lenguaje Python puro.

→ **Django ORM:** Es el "traductor" del framework. Toma el QuerySet de Python y lo convierte automáticamente en sentencias SQL seguras (SELECT, INSERT, UPDATE, DELETE).

→ **SQLite:** El motor de base de datos físico recibe el código SQL, ejecuta la transacción en el archivo db.sqlite3 y devuelve los datos solicitados al ORM.

→ **Template:** La vista toma los datos extraídos de la base de datos y los inyecta dentro de las plantillas HTML para construir una página web dinámica.

→ **Response:** Finalmente, el servidor empaqueta este HTML y lo devuelve al navegador del usuario como una respuesta HTTP (código 200 OK o 302 Redirect), cerrando así el ciclo completo.

Al finalizar el laboratorio, el estudiante deberá ser capaz de diseñar Models, generar y aplicar migraciones, utilizar SQLite como mecanismo de persistencia e implementar operaciones CRUD sin escribir sentencias SQL manualmente. Asimismo, deberá comprender la diferencia entre modificar datos mediante CRUD y modificar la estructura de la base de datos mediante migraciones.

Enunciados

PARTE 1 — Migración del trabajo de la Semana 2 a Django ORM y SQLite

Ejercicio 1 — Recuperar y analizar la aplicación anterior

Abra la aplicación desarrollada en el laboratorio de la Semana 2 e identifique la entidad principal, los datos almacenados en memoria, las Views, URLs, formularios y Templates utilizados. Explique brevemente qué sucede con los registros agregados cuando se reinicia el servidor.

- **Entidad principal:** `Mascota`, compuesta por los campos `nombre`, `especie`, `raza` y `peso`. DOCX
- **Datos almacenados en memoria:** Una lista de diccionarios Python (`mascotas`) definida directamente dentro de `veterinaria/models.py` con 5 registros iniciales. DOCX +1
- **Views utilizadas:**
  - `listar_mascotas`: Obtiene la lista `mascotas` en memoria y la envía al contexto del template de visualización. DOCX
  - `crear_mascota`: Maneja peticiones GET para mostrar el formulario y POST para validar los datos mediante el formulario y agregarlos a la lista en memoria mediante `.append()`. DOCX

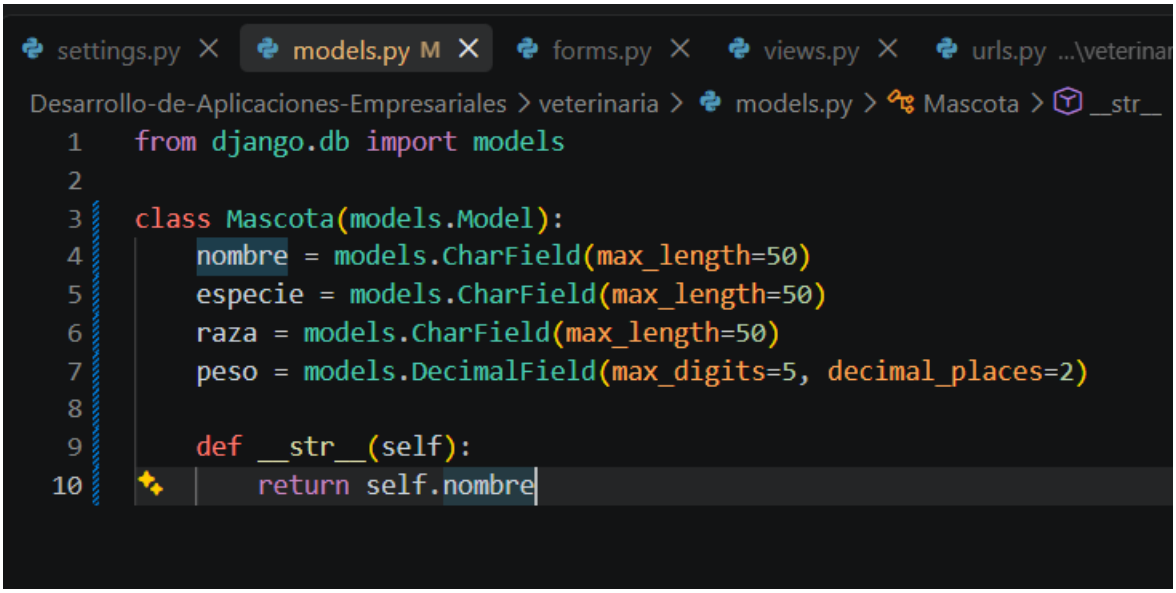
- **URLs configuradas:**
  - `''` (raíz): Vinculada a `listar_mascotas`. DOCX
  - `'crear/'`: Vinculada a `crear_mascota`. DOCX
- **Formulario utilizado:** `MascotaForm` en `veterinaria/forms.py`, basado en `django.forms.Form` con campos `CharField` y `DecimalField`. DOCX
- **Templates utilizados:**
  - `base.html`: Plantilla base con la cabecera, navegación y contenedor principal. DOCX
  - `listar.html`: Renderiza la tabla HTML recorriendo la lista de mascotas con `{% for %}`. DOCX
  - `crear.html`: Renderiza el formulario con `{% csrf_token %}` y campos con `{{ form.as_p }}`. DOCX

COMPORTAMIENTO AL REINICIAR EL SERVIDOR:

Los nuevos registros agregados mediante el formulario se pierden por completo. Esto ocurre porque los datos residen únicamente en la memoria RAM del proceso de Python en ejecución al detener el servidor el estado de la variable “mascotas” se reinicia a su valor inicial definido en el archivo fuente

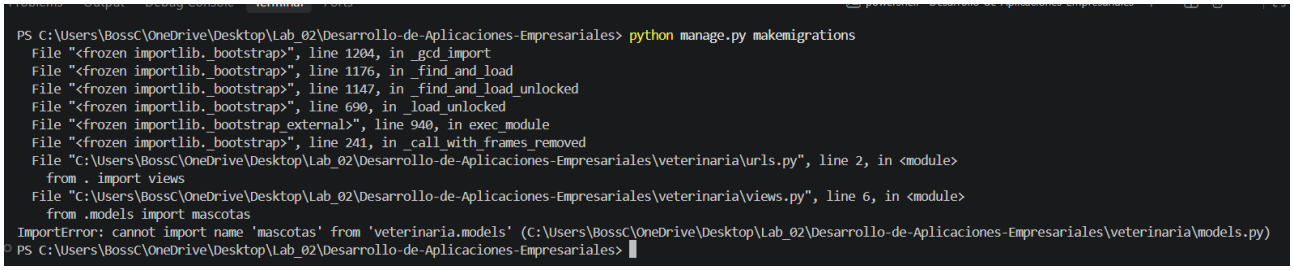
Ejercicio 2 — Convertir la entidad en un Django Model

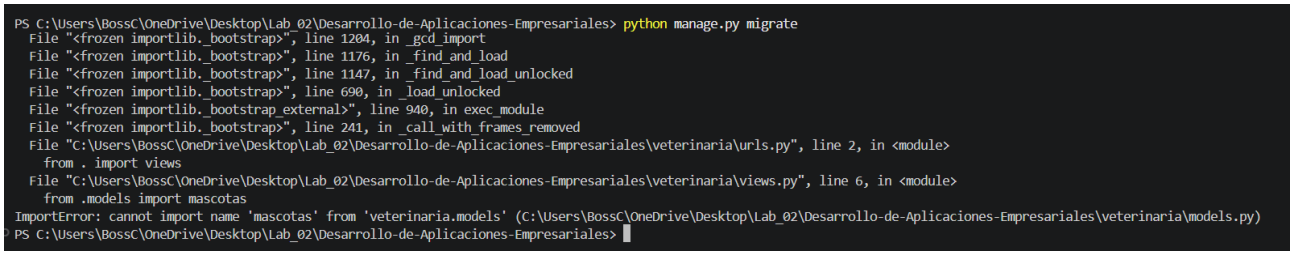
Reemplace la estructura de datos estática utilizada en models.py por un Django Model real. Mantenga los campos definidos a partir de los requisitos del laboratorio anterior y utilice tipos de datos apropiados de Django.

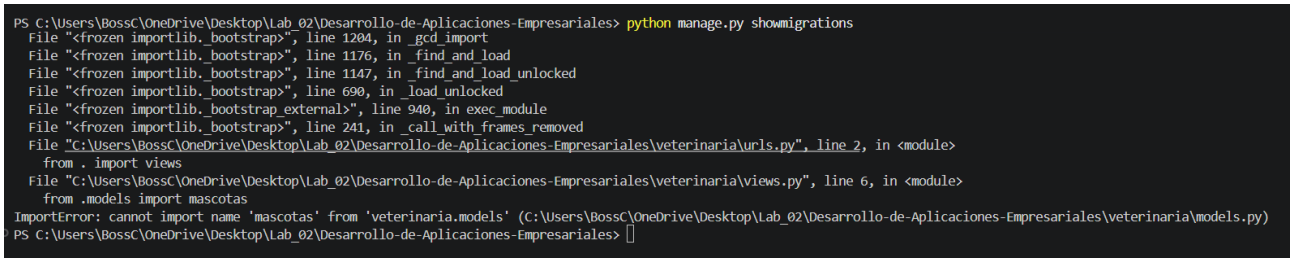


Ejercicio 3 — Crear la estructura persistente de datos

Genere y aplique las migraciones necesarias para crear la tabla correspondiente en SQLite. Documente mediante capturas el resultado de makemigrations, migrate y showmigrations.

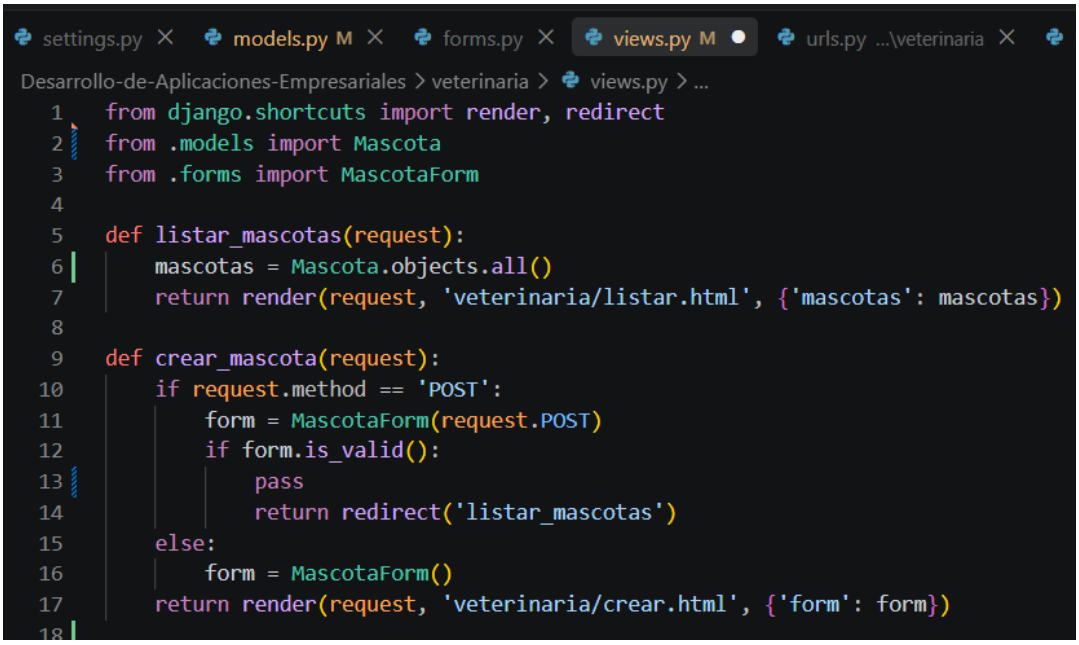






Ejercicio 4 — Implementar la consulta mediante ORM

Modifique la View encargada del listado para que deje de leer los datos desde una lista Python y los obtenga mediante Django ORM. Utilice un QuerySet y verifique que los registros almacenados en SQLite se muestran correctamente en el Template.



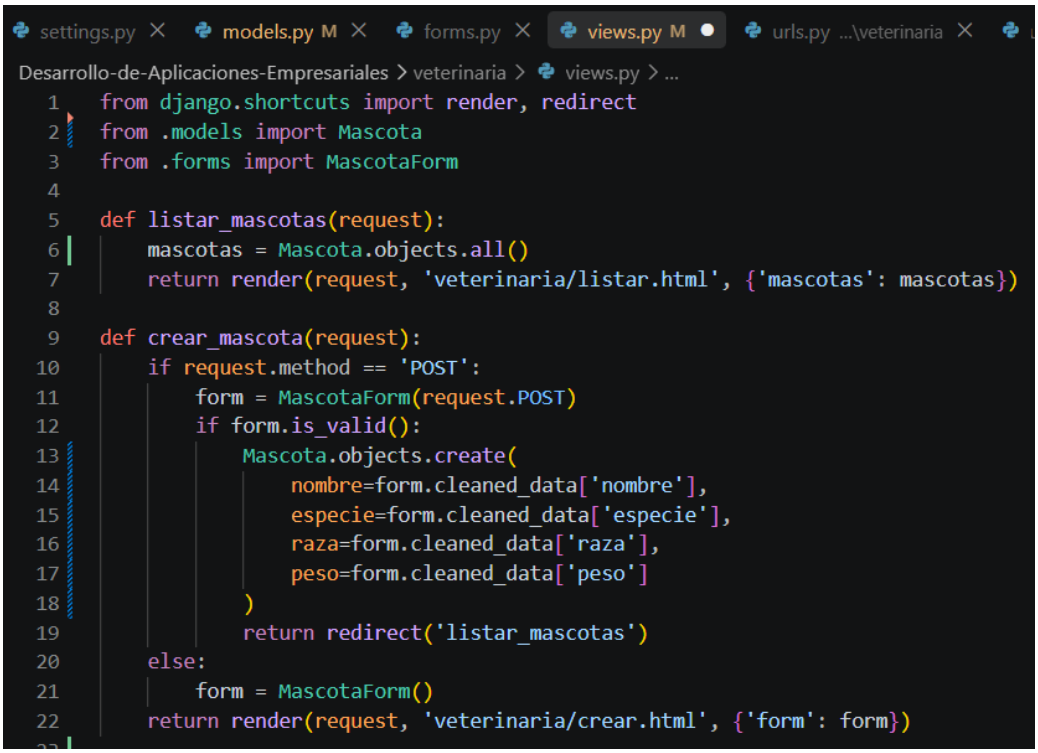

```python
from django.shortcuts import render, redirect
from .models import Mascota
from .forms import MascotaForm

def listar_mascotas(request):
    mascotas = Mascota.objects.all()
    return render(request, 'veterinaria/listar.html', {'mascotas': mascotas})

def crear_mascota(request):
    if request.method == 'POST':
        form = MascotaForm(request.POST)
        if form.is_valid():
            pass
            return redirect('listar_mascotas')
    else:
        form = MascotaForm()
    return render(request, 'veterinaria/crear.html', {'form': form})
```

Ejercicio 5 — Implementar el registro mediante ORM

Adapte la funcionalidad de creación desarrollada en la Semana 2 para que los datos enviados desde el formulario sean almacenados de manera persistente utilizando Django ORM. Después del registro, redirija nuevamente al listado y compruebe que el nuevo dato permanece aunque se reinicie el servidor.




```python
from django.shortcuts import render, redirect
from .models import Mascota
from .forms import MascotaForm

def listar_mascotas(request):
    mascotas = Mascota.objects.all()
    return render(request, 'veterinaria/listar.html', {'mascotas': mascotas})

def crear_mascota(request):
    if request.method == 'POST':
        form = MascotaForm(request.POST)
        if form.is_valid():
            Mascota.objects.create(
                nombre=form.cleaned_data['nombre'],
                especie=form.cleaned_data['especie'],
                raza=form.cleaned_data['raza'],
                peso=form.cleaned_data['peso']
            )
            return redirect('listar_mascotas')
    else:
        form = MascotaForm()
    return render(request, 'veterinaria/crear.html', {'form': form})
```

Ejercicio 6 — Analizar el flujo de persistencia

Documente el recorrido completo de una operación de consulta y una operación de creación:

→ **Request:** El usuario accede en el navegador a la URL raíz mediante una petición HTTP GET.

→ **URL:** El archivo urls.py intercepta la ruta y la enruta a la función listar_mascotas.

→ **View:** views.py solicita la lista completa de registros invocando al modelo.

→ **Model:** La clase Mascota expone su interfaz de datos a través del atributo objects.

→ **Manager / QuerySet:** Se invoca Mascota.objects.all(), generando una estructura de consulta.

→ **Django ORM:** Traduce la sentencia de Python a lenguaje SQL estándar.

→ **SQLite:** El motor de base de datos lee las filas almacenadas en el archivo db.sqlite3 y las retorna.

→ **View:** La vista recibe los registros convertidos en instancias del modelo Mascota.

→ **Context:** Empaqueta los objetos en un diccionario de contexto: {'mascotas': mascotas}.

→ **Template:** No se renderiza una plantilla directa en este punto, sino que se delega al listado.

→ **Response:** Se retorna una respuesta de redirección HTTP 302 Found para que el navegador solicite la vista actualizada.

Identifique además qué operación SQL representa conceptualmente cada acción realizada mediante ORM.

Equivalencia conceptual ORM vs SQL:

- Mascota.objects.all() → **SELECT**
- Mascota.objects.create(...) → **INSERT**
- objeto.save() → **UPDATE**
- objeto.delete() → **DELETE**

PARTE 2 — Investigación y desarrollo de una nueva solución persistente

Ejercicio 7 — Investigar una problemática real

Investigue una problemática real que pueda resolverse mediante una aplicación empresarial web. La problemática seleccionada debe manejar varios tipos de información y permitir justificar la existencia de múltiples entidades dentro de una base de datos. Describa el problema, los usuarios involucrados y el proceso que se desea mejorar.

1. Descripción del problema:

Actualmente, muchas clínicas veterinarias y centros de atención de mascotas gestionan sus operaciones diarias (registro de propietarios, fichas de mascotas, catálogo de medicamentos, servicios clínicos y programación de citas) de forma manual o mediante hojas de cálculo descentralizadas. Esto provoca pérdida de trazabilidad en las atenciones médicas, desorganización en el historial de las mascotas, falta de control en el inventario básico de fármacos y cruce de horarios en la asignación de citas veterinarias.

2. Usuarios involucrados

- **Recepcionista / Personal Administrativo:** Registra nuevos clientes/dueños, da de alta a los pacientes (mascotas) y agenda las citas médicas.
- **Médico Veterinario:** Consulta la información clínica de las mascotas, actualiza estados de atención y revisa la disponibilidad de medicamentos y servicios.
- **Administrador del Centro:** Supervisa los catálogos de servicios, medicamentos disponibles, veterinarios del staff y reportes de atención.

3. Proceso que se desea mejorar

El proceso principal para optimizar es el flujo de registro y control de citas clínicas veterinarias. La plataforma web permitirá centralizar el registro maestro de entidades independientes (como clientes/propietarios, medicamentos y servicios) y vincular de forma persistente y relacional las citas médicas con cada mascota y su respectivo médico tratante, garantizando persistencia en base de datos relacional, integridad referencial y operaciones CRUD inmediatas.

Ejercicio 8 — Definir los requisitos funcionales

Redacte entre 8 y 12 requisitos funcionales derivados de la problemática investigada. Los requisitos deben contemplar como mínimo operaciones para crear, listar, actualizar y eliminar información.

RF01 — Registrar propietario: El sistema debe permitir el registro de nuevos propietarios (dueños) con sus datos personales (nombre, DNI, teléfono, dirección).

RF02 — Listar propietarios: El sistema debe mostrar un catálogo con la información de todos los propietarios registrados.

RF03 — Registrar mascota vinculada: El sistema debe permitir registrar una nueva mascota asignándole obligatoriamente un propietario existente mediante una relación foránea.

RF04 — Consultar y filtrar mascotas: El sistema debe listar todas las mascotas registradas mostrando los datos de su especie, raza, peso y el propietario responsable.

RF05 — Actualizar ficha de mascota: El sistema debe permitir modificar los datos clínicos básicos de una mascota existente (nombre, raza, peso, dueño).

RF06 — Eliminar registro de mascota: El sistema debe permitir la eliminación de una mascota del sistema previa confirmación de la acción.

RF07 — Registrar y listar medicamentos: El sistema debe permitir dar de alta nuevos fármacos en el inventario (nombre comercial, dosis, stock, precio) y visualizarlos en una tabla.

RF08 — Gestionar catálogo de servicios: El sistema debe permitir crear, listar, actualizar y eliminar los servicios clínicos ofrecidos (nombre del servicio, costo, duración estimada).

RF09 — Registrar veterinarios: El sistema debe permitir el alta y consulta del personal médico veterinario (nombre, especialidad, número de colegiatura y horario).

RF10 — Mantener integridad referencial: El sistema debe garantizar que no se registren mascotas sin un propietario válido y gestionar las políticas de eliminación en cascada o protección de datos.

Ejercicio 9 — Diseñar el modelo de datos
Identifique y diseñe como mínimo cinco entidades:

Para cada entidad defina:

- nombre;
- campos;
- tipos de datos;
- obligatoriedad;
- clave primaria;
- clave foránea cuando corresponda.
- Justifique por qué cada entidad y cada relación son necesarias para resolver la problemática.

Tres entidades independientes o no relacionadas entre sí;

Entidad: Medicamento

- id: Clave primaria autoincremental (AutoField / BigAutoField, Obligatorio).
- nombre: CharField(max_length=100) (Obligatorio).
- descripcion: CharField(max_length=200) (Obligatorio).
- stock: IntegerField (Obligatorio).
- precio: DecimalField(max_digits=6, decimal_places=2) (Obligatorio).
- Justificación: Permite mantener el control de inventario y costos de fármacos e insumos médicos disponibles en la clínica sin depender de otros módulos.

Entidad: Servicio

- id: Clave primaria autoincremental (AutoField / BigAutoField, Obligatorio).
- nombre: CharField(max_length=100) (Obligatorio).
- duracion_minutos: IntegerField (Obligatorio).
- precio: DecimalField(max_digits=6, decimal_places=2) (Obligatorio).
- Justificación: Gestiona el catálogo de prestaciones que ofrece el centro (baño, desparasitación, cirugía, consulta general) con su tarifa y tiempo estimado.

Entidad: Veterinario

- id: Clave primaria autoincremental (AutoField / BigAutoField, Obligatorio).
- nombre_completo: CharField(max_length=100) (Obligatorio).
- especialidad: CharField(max_length=80) (Obligatorio).
- colegiatura: CharField(max_length=20) (Obligatorio).
- *Justificación:* Almacena el registro oficial del staff médico responsable de las atenciones y prescripciones.

Dos entidades que mantengan una relación mediante ForeignKey.

Entidad 1 (Principal): Propietario

- id: Clave primaria autoincremental (AutoField / BigAutoField, Obligatorio).
- nombres: CharField(max_length=80) (Obligatorio).
- apellidos: CharField(max_length=80) (Obligatorio).
- dni: CharField(max_length=8) (Obligatorio, único).
- telefono: CharField(max_length=15) (Obligatorio).
- direccion: CharField(max_length=150) (Obligatorio).
- *Justificación:* Modela al cliente o tutor responsable legal y financiero de los pacientes atendidos en la clínica.

Entidad 2 (Dependiente): Paciente (Mascota)

- id: Clave primaria autoincremental (AutoField / BigAutoField, Obligatorio).
- propietario: ForeignKey(Propietario, on_delete=models.CASCADE) (Clave Foránea, Obligatorio).
- nombre: CharField(max_length=50) (Obligatorio).
- especie: CharField(max_length=30) (Obligatorio).
- raza: CharField(max_length=50) (Obligatorio).
- edad: IntegerField (Obligatorio).
- peso: DecimalField(max_digits=5, decimal_places=2) (Obligatorio).
- *Justificación:* Representa al animal que recibe el tratamiento clínico.

Justificación de la Relación 1 a N (ForeignKey): Un Propietario puede tener registradas una o varias mascotas (Pacientes), pero cada mascota pertenece estrictamente a un único tutor responsable. La clave foránea en Paciente garantiza la integridad referencial y permite asociar el historial del animal a los datos de contacto del dueño en caso de emergencias.

Ejercicio 10 — Representar las relaciones

Elabore un diagrama sencillo del modelo de datos donde se identifiquen claramente las cinco entidades y la relación entre las dos entidades vinculadas.

Ejemplo conceptual:

Entidad A

Entidad B

Entidad C

Entidad D

1

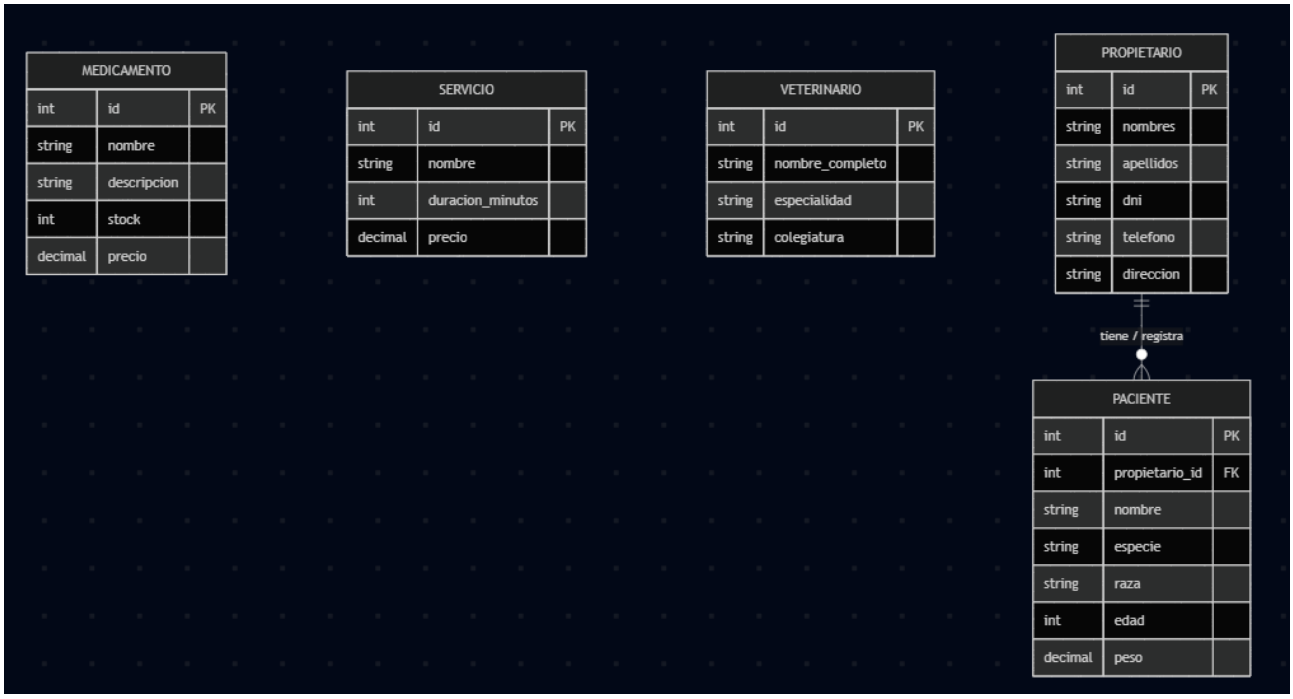
|

|

N

Entidad E

La problemática, los nombres de las entidades y los campos deben ser definidos por el estudiante a partir de su investigación.



Entidades independientes: Medicamento, Servicio y Veterinario operan de forma autónoma sin claves foráneas.

Entidades relacionadas: Propietario (Entidad 1) se relaciona con Paciente (Entidad N) mediante el campo propietario_id como clave foránea con comportamiento CASCADE.

Ejercicio 11 — Crear la aplicación Django

Cree una nueva App Django para implementar la problemática seleccionada. Regístrela correctamente en INSTALLED_APPS y configure sus URLs dentro del Project.

```
32
33  INSTALLED_APPS = [
34      'django.contrib.admin',
35      'django.contrib.auth',
36      'django.contrib.contenttypes',
37      'django.contrib.sessions',
38      'django.contrib.messages',
39      'django.contrib.staticfiles',
40      'veterinaria',
41      'gestion_clinica',  # <-- Nueva App agregada
42  ]
```

settings.py M | urls.py ...\veterinaria M | urls.py ...\gestion_clinica U

Desarrollo-de-Aplicaciones-Empresariales > veterinaria > urls.py > ...

```
1  from django.contrib import admin
2  from django.urls import path, include
3
4  urlpatterns = [
5      path('admin/', admin.site.urls),
6      path('', include('veterinaria.urls')),
7      path('clinica/', include('gestion_clinica.urls')),  # <-- Rutas de la nueva App
8  ]
```

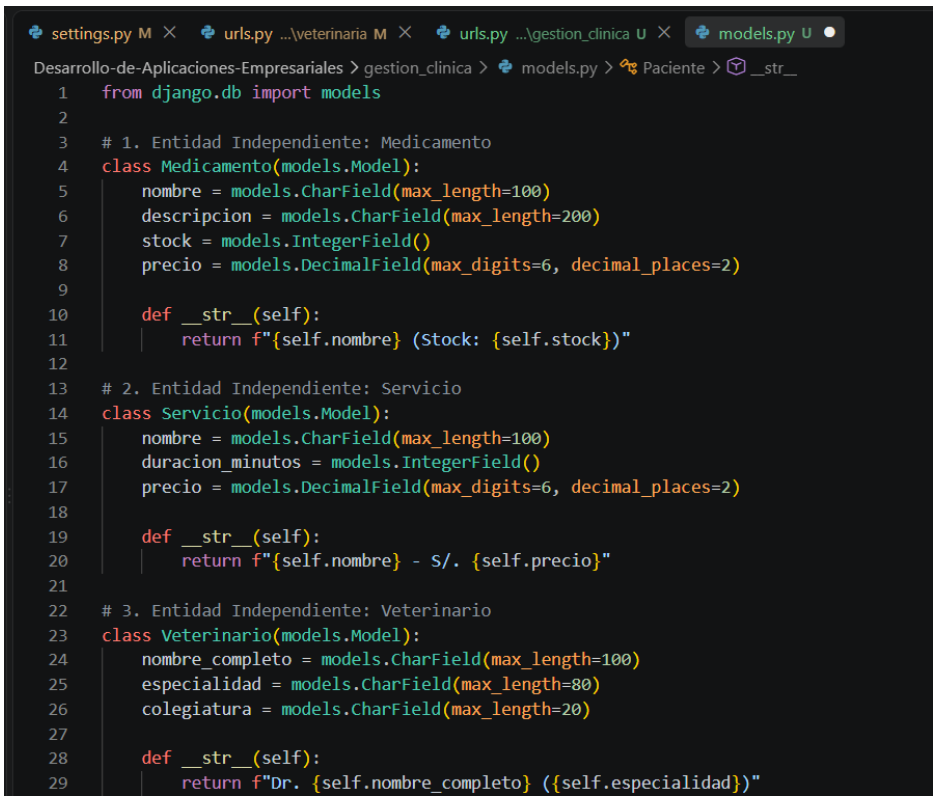
settings.py M | urls.py ...\veterinaria M | urls.py ...\gestion_clinica U

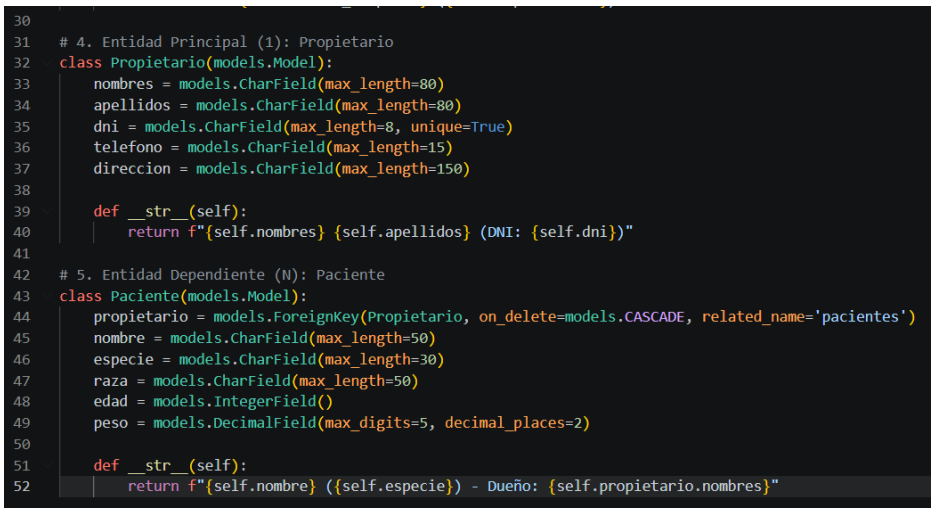
Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > urls.py > ...

```
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      # Las rutas CRUD las agregaremos en los siguientes ejercicios
6  ]
```

Ejercicio 12 — Implementar los Models

Traduzca el diseño del Ejercicio 9 a Django Models. Implemente las cinco entidades y configure correctamente la relación ForeignKey entre las dos entidades relacionadas.





Ejercicio 13 — Crear la base de datos mediante migraciones

Ejecute las migraciones necesarias para crear la estructura diseñada en SQLite:

python manage.py makemigrations

python manage.py migrate

Verifique posteriormente su estado mediante:

python manage.py showmigrations

```
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales> python manage.py makemigrations
Migrations for 'gestion_clinica':
  gestion_clinica\migrations\0001_initial.py
    + Create model Medicamento
    + Create model Propietario
    + Create model Servicio
    + Create model Veterinario
    + Create model Paciente
```

```
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion_clinica, sessions, veterinaria
Running migrations:
  Applying gestion_clinica.0001_initial... OK
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales>
```

```
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales> python manage.py showmigrations
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
gestion_clinica
 [X] 0001_initial
sessions
 [X] 0001_initial
veterinaria
 [X] 0001_initial
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales>
```

Ejercicio 14 — Implementar CREATE

Implemente una funcionalidad web que permita registrar nuevos datos para las entidades mediante Django ORM. Después de guardar un registro, la aplicación debe redirigir al listado correspondiente.

settings.py M | urls.py ...\veterinaria | forms.py U | views.py U | urls.py ...\gestion_clinica U

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > forms.py > PacienteForm > Meta

```python
from django import forms
from .models import Propietario, Paciente, Medicamento, Servicio, Veterinario

class PropietarioForm(forms.ModelForm):
    class Meta:
        model = Propietario
        fields = ['nombres', 'apellidos', 'dni', 'telefono', 'direccion']

class PacienteForm(forms.ModelForm):
    class Meta:
        model = Paciente
        fields = ['propietario', 'nombre', 'especie', 'raza', 'edad', 'peso']
```

settings.py M | urls.py ...\veterinaria | forms.py U | views.py U | urls.py ...\gestion_clinica U

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > views.py > crear_paciente

```python
from django.shortcuts import render, redirect
from .models import Propietario, Paciente
from .forms import PropietarioForm, PacienteForm

def listar_pacientes(request):
    pacientes = Paciente.objects.select_related('propietario').all()
    propietarios = Propietario.objects.all()
    return render(request, 'gestion_clinica/listar.html', {
        'pacientes': pacientes,
        'propietarios': propietarios
    })

def crear_propietario(request):
    if request.method == 'POST':
        form = PropietarioForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('listar_clinica')
    else:
        form = PropietarioForm()
    return render(request, 'gestion_clinica/crear_propietario.html', {'form': form})

def crear_paciente(request):
    if request.method == 'POST':
        form = PacienteForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('listar_clinica')
    else:
        form = PacienteForm()
    return render(request, 'gestion_clinica/crear_paciente.html', {'form': form})
```

settings.py M | urls.py ...\veterinaria | forms.py U | views.py U | urls.py ...\gestion_clinica U

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > urls.py > ...

```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.listar_pacientes, name='listar_clinica'),
    path('propietario/nuevo/', views.crear_propietario, name='crear_propietario'),
    path('paciente/nuevo/', views.crear_paciente, name='crear_paciente'),
]
```

HTML:

forms.py U | views.py U | urls.py ...\gestion_clinica U | crear_propietario.html U | crear_paciente.html U

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > templates > gestion_clinica > crear_paciente.html > html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Registrar Paciente</title>
</head>
<body>
    <h2>Registrar Nuevo Paciente (Mascota)</h2>
    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Guardar Paciente</button>
    </form>
    <br>
    <a href="{% url 'listar_clinica' %}">Volver al listado</a>
</body>
</html>
```

forms.py U | views.py U | urls.py ...\gestion_clinica U | crear_propietario.html

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > templates > gestion_clinica > cr

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Registrar Propietario</title>
</head>
<body>
    <h2>Registrar Nuevo Propietario</h2>
    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Guardar Propietario</button>
    </form>
    <br>
    <a href="{% url 'listar_clinica' %}">Volver al listado</a>
</body>
</html>
```

forms.py U × views.py U × urls.py ...\gestion_clinica U × crear_propietario.html U × crear_paciente.html U ×

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > templates > gestion_clinica > listar.html > html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Gestión Clínica</title>
</head>
<body>
    <h1>Sistema de Gestión Clínica Veterinaria</h1>
    <a href="{% url 'crear_propietario' %}">+ Registrar Propietario</a> |
    <a href="{% url 'crear_paciente' %}">+ Registrar Paciente</a>

    <h2>Propietarios Registrados</h2>
    <table border="1">
        <tr>
            <th>Nombre Completo</th>
            <th>DNI</th>
            <th>Teléfono</th>
            <th>Dirección</th>
        </tr>
        {% for prop in propietarios %}
        <tr>
            <td>{{ prop.nombres }} {{ prop.apellidos }}</td>
            <td>{{ prop.dni }}</td>
            <td>{{ prop.telefono }}</td>
            <td>{{ prop.direccion }}</td>
        </tr>
        {% empty %}
        <tr><td colspan="4">No hay propietarios registrados.</td></tr>
        {% endfor %}
    </table>

    <h2>Pacientes Registrados (Relación 1:N)</h2>
    <table border="1">
        <tr>
            <th>Nombre</th>
            <th>Especie</th>
            <th>Raza</th>
            <th>Edad</th>
            <th>Peso (kg)</th>
            <th>Propietario Responsable</th>
        </tr>
        {% for pac in pacientes %}
        <tr>
            <td>{{ pac.nombre }}</td>
            <td>{{ pac.especie }}</td>
            <td>{{ pac.raza }}</td>
            <td>{{ pac.edad }}</td>
            <td>{{ pac.peso }}</td>
            <td>{{ pac.propietario.nombres }} {{ pac.propietario.apellidos }}</td>
        </tr>
        {% empty %}
        <tr><td colspan="6">No hay pacientes registrados.</td></tr>
        {% endfor %}
    </table>
</body>
</html>
```

RESULTADO:



Conceptualmente:

→ **Formulario:** El usuario rellena los campos en la interfaz web y pulsa el botón de envío.

→ **POST:** El navegador empaqueta los datos y despacha una solicitud HTTP POST hacia la ruta configurada.

→ **View:** La función en views.py (crear_paciente o crear_propietario) recibe la petición, instancia el formulario con request.POST y ejecuta la validación con form.is_valid().

→ **ORM:** Al invocarse form.save(), Django ORM procesa los datos limpios, resuelve la clave foránea (propietario_id) y construye la consulta SQL parametrizada.

→ **INSERT:** Se genera y ejecuta la instrucción SQL de inserción:

INSERT INTO gestion_clinica_paciente (propietario_id, nombre, especie, raza, edad, peso)

VALUES (1, 'Valky', 'Perro', 'Bulldog Francés', 2, 12.50);

→ **SQLite:** El motor escribe físicamente la nueva fila en el archivo db.sqlite3 y le asigna una clave primaria (id) autoincremental.

→ **redirect:** La vista responde con una cabecera de redirección HTTP (302 Found) invocando redirect('listar_clinica').

→ **listado:** El navegador realiza automáticamente una nueva petición GET hacia la URL principal, donde la vista ejecuta la consulta y renderiza la tabla con el registro actualizado.

Ejercicio 15 — Implementar READ

Implemente una pantalla de listado para cada entidad utilizando QuerySets. Los datos deberán ser obtenidos desde SQLite mediante Django ORM y mostrados mediante Templates.

Se espera utilizar operaciones como:

Model.objects.all()

Model.objects.filter(...)

Model.objects.order_by(...)

cuando sean necesarias según el problema investigado.

```python
def listar_pacientes(request):
    pacientes = Paciente.objects.all()
    propietarios = Propietario.objects.order_by('apellidos')
    medicamentos = Medicamento.objects.filter(stock__gt=0)
    servicios = Servicio.objects.order_by('precio')
    veterinarios = Veterinario.objects.all()

    contexto = {
        'pacientes': pacientes,
        'propietarios': propietarios,
        'medicamentos': medicamentos,
        'servicios': servicios,
        'veterinarios': veterinarios
    }
    return render(request, 'gestion_clinica/listar.html', contexto)
```

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Gestión Clínica</title>
</head>
<body>
    <h1>Sistema de Gestión Clínica Veterinaria</h1>

    <a href="{% url 'crear_propietario' %}">+ Registrar Propietario</a> |
    <a href="{% url 'crear_paciente' %}">+ Registrar Paciente</a>

    <h2>Pacientes Registrados (all)</h2>
    <table border="1">
        <tr>
            <th>Nombre</th>
            <th>Especie</th>
            <th>Raza</th>
            <th>Propietario Responsable</th>
        </tr>
        {% for pac in pacientes %}
        <tr>
            <td>{{ pac.nombre }}</td>
            <td>{{ pac.especie }}</td>
            <td>{{ pac.raza }}</td>
            <td>{{ pac.propietario.nombres }} {{ pac.propietario.apellidos }}</td>
        </tr>
        {% endfor %}
    </table>

    <h2>Propietarios (order_by apellidos)</h2>
    <table border="1">
        <tr>
            <th>Apellidos</th>
            <th>Nombres</th>
            <th>DNI</th>
```

```
            <th>Teléfono</th>
        </tr>
        {% for prop in propietarios %}
        <tr>
            <td>{{ prop.apellidos }}</td>
            <td>{{ prop.nombres }}</td>
            <td>{{ prop.dni }}</td>
            <td>{{ prop.telefono }}</td>
        </tr>
        {% endfor %}
    </table>

    <h2>Medicamentos Disponibles (filter stock > 0)</h2>
    <table border="1">
        <tr>
            <th>Nombre</th>
            <th>Stock</th>
            <th>Precio</th>
        </tr>
        {% for med in medicamentos %}
        <tr>
            <td>{{ med.nombre }}</td>
            <td>{{ med.stock }}</td>
            <td>S/ {{ med.precio }}</td>
        </tr>
        {% endfor %}
    </table>

    <h2>Servicios Clínicos (order_by precio)</h2>
    <table border="1">
        <tr>
```

```
            <th>Servicio</th>
            <th>Duración (min)</th>
            <th>Precio</th>
        </tr>
        {% for serv in servicios %}
        <tr>
            <td>{{ serv.nombre }}</td>
            <td>{{ serv.duracion_minutos }}</td>
            <td>S/ {{ serv.precio }}</td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>
```

RESULTADO:

Sistema de Gestión Clínica Veterinaria

+ Registrar Propietario | + Registrar Paciente

Pacientes Registrados (all)

Nombre	Especie	Raza	Propietario Responsable
Valky	Perro	Bulldog Frances	Jesus Jose Castillo Sumire

Propietarios (order_by apellidos)

Apellidos	Nombres	DNI	Teléfono
Castillo Sumire	Jesus Jose	71541966	920434999

Medicamentos Disponibles (filter stock > 0)

Nombre	Stock	Precio

Servicios Clínicos (order_by precio)

Servicio	Duración (min)	Precio

Ejercicio 16 — Implementar UPDATE

Cada registro deberá disponer de una opción para actualizar su información. La View deberá recuperar el objeto almacenado, modificar sus propiedades y guardar los cambios mediante Django ORM.

forms.py U | views.py U | urls.py ...\gestion_clinica U | editar_paciente.html U | crear_propietario.html U | crear_pac

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > views.py > editar_paciente

```python
def editar_paciente(request, id):
    # Recupera el objeto almacenado o devuelve error 404 si no existe
    paciente = get_object_or_404(Paciente, id=id)

    if request.method == 'POST':
        # Instancia el formulario con los datos enviados y el objeto existente
        form = PacienteForm(request.POST, instance=paciente)
        if form.is_valid():
            form.save()  # ORM ejecuta el UPDATE en SQLite
            return redirect('listar_clinica')
    else:
        # Petición GET: Carga el formulario con los datos actuales
        form = PacienteForm(instance=paciente)

    return render(request, 'gestion_clinica/editar_paciente.html', {'form': form, 'paciente': paciente})
```

forms.py U | views.py U | urls.py ...\gestion_clinica U | editar_paciente.html U | crear_propietari

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > urls.py > ...

```python
    # Nueva ruta para UPDATE
    path('paciente/editar/<int:id>/', views.editar_paciente, name='editar_paciente'),
]
```

forms.py U | views.py U | urls.py ...\gestion_clinica U | editar_paciente.html U | crear_pr

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > templates > gestion_clinica > editar_paciente.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Editar Paciente</title>
</head>
<body>
    <h2>Editar Paciente: {{ paciente.nombre }}</h2>

    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Actualizar Datos</button>
    </form>

    <br>
    <a href="{% url 'listar_clinica' %}">Cancelar y volver al listado</a>
</body>
</html>
```

```
13  <h2>Pacientes Registrados (all)</h2>
14      <table border="1">
15          <tr>
16              <th>Nombre</th>
17              <th>Especie</th>
18              <th>Raza</th>
19              <th>Propietario Responsable</th>
20              <th>Acciones</th> <!-- Nueva cabecera -->
21          </tr>
22          {% for pac in pacientes %}
23          <tr>
24              <td>{{ pac.nombre }}</td>
25              <td>{{ pac.especie }}</td>
26              <td>{{ pac.raza }}</td>
27              <td>{{ pac.propietario.nombres }} {{ pac.propietario.apellidos }}</td>
28              <td>
29                  <!-- Nuevo enlace de edición -->
30                  <a href="{% url 'editar_paciente' pac.id %}">Editar</a>
31              </td>
32          </tr>
33          {% endfor %}
34      </table>
```

RESULTADO:

Sistema de Gestión Clínica Veterinaria

+ Registrar Propietario | + Registrar Paciente

Pacientes Registrados (all)

Nombre	Especie	Raza	Propietario Responsable	Acciones
Valky	Perro	Bulldog Frances	Jesus Jose Castillo Sumire	Editar

Propietarios (order_by apellidos)

Apellidos	Nombres	DNI	Teléfono
Castillo Sumire	Jesus Jose	71541966	920434999

Medicamentos Disponibles (filter stock > 0)

Nombre	Stock	Precio

Servicios Clínicos (order_by precio)

Servicio	Duración (min)	Precio

Editar Paciente: Valky

Propietario: Jesus Jose Castillo Sumire (DNI: 71541966)

Nombre: Valky

Especie: Perro

Raza: Bulldog Frances

Edad: 3

Peso: 12.50

Actualizar Datos

Cancelar y volver al listado

Conceptualmente:

→ **Seleccionar registro:** El usuario hace clic en el enlace de edición de una fila específica, enviando el identificador (id) del registro a través de la URL.

→ **GET:** El navegador emite una petición HTTP GET hacia la ruta de edición.

→ **Formulario con datos actuales:** La vista consulta la base de datos para obtener el objeto exacto (usando get_object_or_404). Luego, instancia el formulario vinculándolo a ese objeto, lo que genera un HTML con los campos prellenados con la información actual.

→ **POST:** El usuario modifica los datos deseados (por ejemplo, corrige el nombre o el peso) y presiona el botón de guardar, despachando una petición HTTP POST.

→ **Modificar objeto, save()::** La vista asocia los nuevos datos recibidos al objeto existente. Tras verificar que son válidos con form.is_valid(), invoca el método form.save().

→ **ORM:** Django ORM evalúa el objeto y, al detectar que ya posee una clave primaria (id) asignada, comprende que no debe crear un registro nuevo, sino modificar uno existente.

→ **UPDATE:** El ORM traduce la acción a una sentencia SQL de actualización. Conceptualmente:

UPDATE gestion_clinica_paciente SET peso = 14.20 WHERE id = 1;

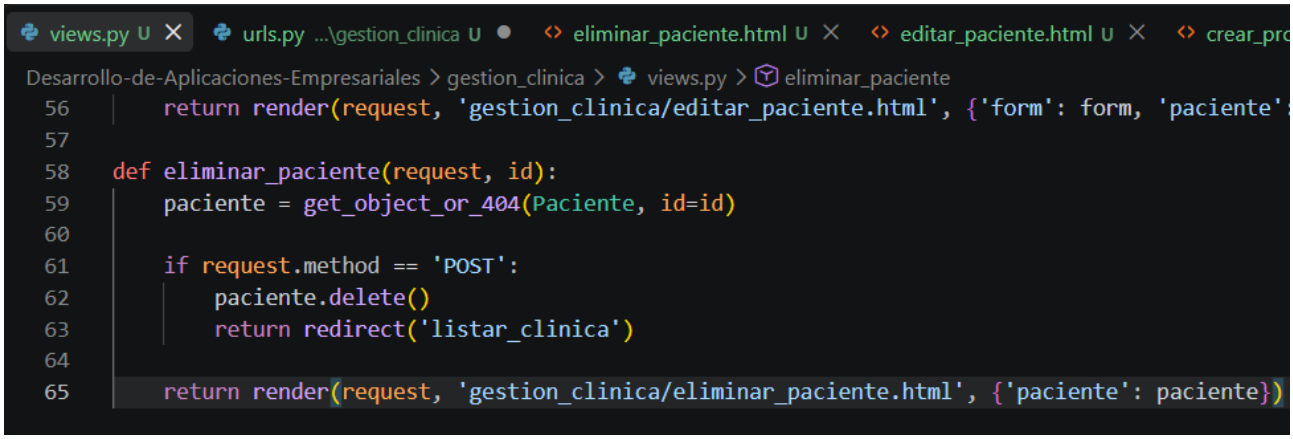
→ **SQLite:** El motor de base de datos ubica la fila correspondiente en el archivo db.sqlite3, sobrescribe los valores y confirma la transacción.

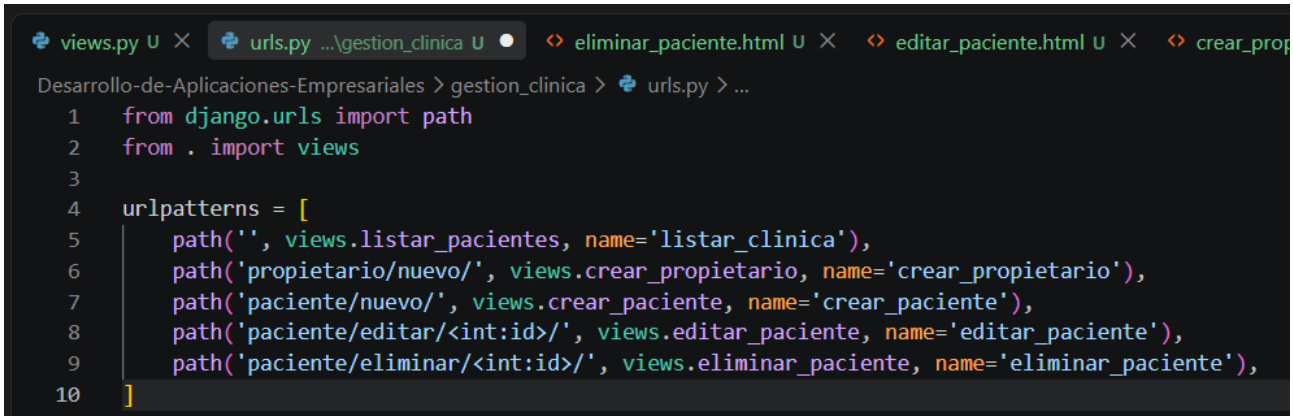
→ **Redirect:** La vista responde al navegador con un estado HTTP 302 Found, instruyéndolo a redirigirse a la URL del listado de la clínica.

→ **Listado actualizado:** El navegador ejecuta una nueva petición GET a la vista principal. El ORM vuelve a consultar la tabla completa y el *Template* se renderiza mostrando las modificaciones reflejadas en la pantalla.

Ejercicio 17 — Implementar DELETE

Cada registro deberá disponer de una opción de eliminación. La aplicación deberá mostrar primero una confirmación y realizar la eliminación mediante una petición POST.





views.py U | urls.py ...\gestion_clinica U | eliminar_paciente.html U | editar_paciente.html U

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > templates > gestion_clinica > eliminar_paciente.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Eliminar Paciente</title>
</head>
<body>
    <h2>¿Estás seguro de que deseas eliminar al paciente {{ paciente.nombre }}?</h2>

    <p>Esta acción no se puede deshacer.</p>

    <form method="POST">
        {% csrf_token %}
        <button type="submit">Sí, eliminar paciente</button>
    </form>

    <br>
    <a href="{% url 'listar_clinica' %}">Cancelar y volver al listado</a>
</body>
</html>
```

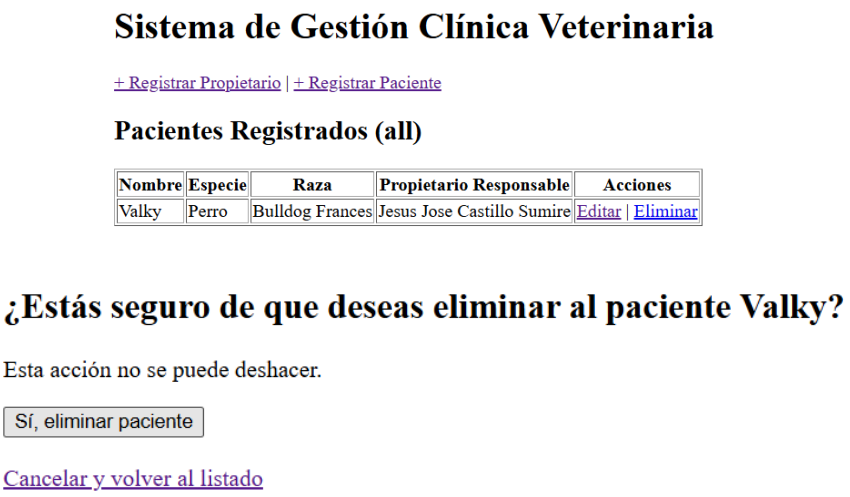
views.py U | urls.py ...\gestion_clinica U | eliminar_paciente.html U | editar_paciente.html U

Desarrollo-de-Aplicaciones-Empresariales > gestion_clinica > templates > gestion_clinica > listar.html > html

```
<html lang="es">
<body>
    <a href="{% url 'crear_paciente' %}">+ Registrar Paciente</a>

<h2>Pacientes Registrados (all)</h2>
    <table border="1">
        <tr>
            <th>Nombre</th>
            <th>Especie</th>
            <th>Raza</th>
            <th>Propietario Responsable</th>
            <th>Acciones</th>
        </tr>
        {% for pac in pacientes %}
        <tr>
            <td>{{ pac.nombre }}</td>
            <td>{{ pac.especie }}</td>
            <td>{{ pac.raza }}</td>
            <td>{{ pac.propietario.nombres }} {{ pac.propietario.apellidos }}</td>
            <td>
                <a href="{% url 'editar_paciente' pac.id %}">Editar</a> |
                <a href="{% url 'eliminar_paciente' pac.id %}">Eliminar</a>
            </td>
        </tr>
        {% endfor %}
    </table>
```

RESULTADO:



Conceptualmente:

→ **Seleccionar registro:** El usuario hace clic en el enlace "Eliminar" correspondiente a una fila, enviando el identificador (id) del registro a través de la URL.

→ **Confirmación:** La vista procesa una petición GET y renderiza una plantilla HTML intermedia para preguntar al usuario si realmente desea borrar la información, evitando eliminaciones accidentales.

→ **POST:** Al presionar el botón de confirmación, el formulario emite una petición HTTP segura de tipo POST hacia el servidor.

→ **Delete():** La vista de Django recupera el objeto específico y ejecuta el método paciente.delete() sobre esa instancia.

→ **ORM:** El mapeador objeto-relacional de Django intercepta el método de Python y se encarga de traducirlo a una operación de base de datos.

→ **DELETE:** El ORM construye la sentencia SQL correspondiente (conceptualmente: DELETE FROM gestion_clinica_paciente WHERE id = X;).

→ **SQLite:** El motor de la base de datos recibe la instrucción y borra físicamente la fila del archivo db.sqlite3.

→ **Redirect:** La función de la vista termina devolviendo un estado HTTP 302, lo que obliga al navegador a redirigirse automáticamente a la URL del listado general.

→ **Listado actualizado:** El navegador emite una nueva petición GET hacia la vista principal. El ORM vuelve a hacer un SELECT de los datos y el HTML se dibuja mostrando la tabla sin el registro que acaba de ser eliminado.

Ejercicio 18 — Verificar las entidades relacionadas

Realice pruebas que demuestren que la relación entre las dos entidades funciona correctamente. Registre datos asociados, consulte registros relacionados y compruebe la clave foránea generada por Django.

```
PS C:\Users\BossC\OneDrive\Desktop\Lab_02> cd Desarrollo-de-Aplicaciones-Empresariales
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales> python manage.py shell
12 objects imported automatically (use -v 2 for details).

Ctrl click to launch VS Code Native REPL
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

```
Ctrl click to launch VS Code Native REPL
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from gestion_clinica.models import Propietario, Paciente
>>> mascota = Paciente.objects.first()
>>> print(f"Mascota: {mascota.nombre}")
Mascota: Valky
>>> print(f"Valor de la clave foránea (propietario_id): {mascota.propietario_id}")
Valor de la clave foránea (propietario_id): 1
>>> dueño = mascota.propietario
>>> print(f"El dueño de {mascota.nombre} es {dueño.nombres} {dueño.apellidos}")
El dueño de Valky es Jesus Jose Castillo Sumire
>>> mascotas_del_dueño = dueño.pacientes.all()
>>> print(f"Mascotas registradas a nombre de {dueño.nombres}: {mascotas_del_dueño}")
Mascotas registradas a nombre de Jesus Jose: <QuerySet [<Paciente: Valky (Perro) - Dueño: Jesus Jose>]>
>>>
```

Ejercicio 19 — Verificar el CRUD completo

Para cada entidad implementada, compruebe las operaciones:

```
>>> from gestion_clinica.models import Medicamento
>>>
>>> nuevo_med = Medicamento(nombre="Amoxicilina Canina", descripcion="Antibiotico", stock=20, precio=25.50)
>>> nuevo_med.save()
>>> print(f"CREATE (INSERT) completado: {nuevo_med.nombre}")
CREATE (INSERT) completado: Amoxicilina Canina
>>>
>>> med_leido = Medicamento.objects.get(id=nuevo_med.id)
>>> print(f"READ (SELECT) completado: {med_leido.nombre} con stock {med_leido.stock}")
READ (SELECT) completado: Amoxicilina Canina con stock 20
>>>
>>> med_leido.stock = 15
>>> med_leido.save()
>>> print(f"UPDATE completado: El nuevo stock es {med_leido.stock}")
UPDATE completado: El nuevo stock es 15
>>>
>>> med_leido.delete()
(1, {'gestion_clinica.Medicamento': 1})
>>> print("DELETE completado: Registro borrado de SQLite exitosamente")
DELETE completado: Registro borrado de SQLite exitosamente
>>>
```

CREATE → INSERT

- Al instanciar Medicamento(...) y llamar a save(), el ORM construye y ejecuta un INSERT en SQLite.

READ → SELECT

- Al usar Medicamento.objects.get(...), el ORM lanza un SELECT a la base de datos para recuperar la fila exacta.

UPDATE → UPDATE

- Al modificar la propiedad med_leido.stock = 15 y volver a llamar a save(), el ORM detecta que el objeto ya existe y ejecuta un UPDATE en lugar de un insert.

DELETE → DELETE

- Al invocar med_leido.delete(), el ORM envía la orden DELETE a SQLite, borrando el registro de forma permanente.

Todas las operaciones deben realizarse mediante Django ORM y SQLite, sin utilizar sentencias SQL manuales.

```
>>>
>>> exit()
PS C:\Users\BossC\OneDrive\Desktop\Lab_02\Desarrollo-de-Aplicaciones-Empresariales>
```

Ejercicio 20 — Documentar el flujo completo

Seleccione al menos una de las entidades y documente mediante capturas el recorrido de las cuatro operaciones CRUD:

Flujo CREATE (Registrar un Paciente):



→ **Navegador:** El usuario llena el formulario web y hace clic en guardar, enviando los datos mediante una petición POST.

→ **URL:** La ruta paciente/nuevo/ configurada en urls.py recibe e intercepta la petición.

→ **View:** La función crear_paciente recibe los datos y valida el formulario.

→ **Model:** El formulario se vincula directamente a la estructura de la clase Paciente.

→ **Django ORM:** Al ejecutar form.save(), el ORM prepara la sentencia de inserción.

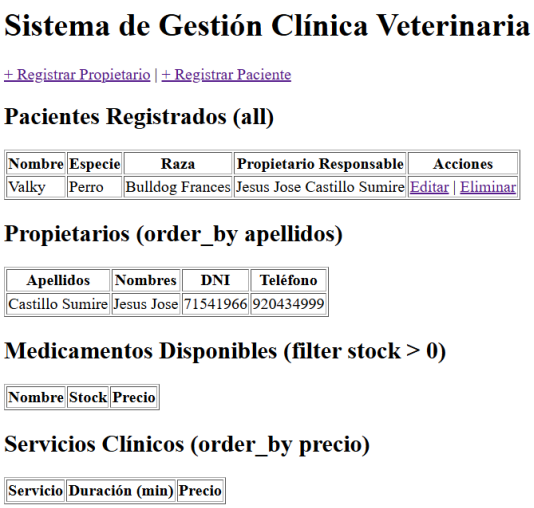
→ **SQLite:** Recibe la instrucción INSERT y guarda la nueva fila físicamente en la base de datos.

→ **Redirect / Render:** La vista finaliza respondiendo con un redirect (redirección) hacia la página principal.

→ **Template:** No se renderiza plantilla aquí, se delega a la vista del listado.

→ **Navegador:** Recibe la orden 302 y carga la nueva página actualizada.

Flujo READ (Listar Pacientes):



Sistema de Gestión Clínica Veterinaria

+ Registrar Propietario | + Registrar Paciente

Pacientes Registrados (all)

Nombre	Especie	Raza	Propietario Responsable	Acciones
Valky	Perro	Bulldog Frances	Jesus Jose Castillo Sumire	Editar | Eliminar

Propietarios (order_by apellidos)

Apellidos	Nombres	DNI	Teléfono
Castillo Sumire	Jesus Jose	71541966	920434999

Medicamentos Disponibles (filter stock > 0)

Nombre	Stock	Precio

Servicios Clínicos (order_by precio)

Servicio	Duración (min)	Precio

→ **Navegador:** El usuario ingresa a la dirección web y emite una petición GET.

→ **URL:** La ruta raíz de la aplicación dirige el tráfico.

→ **View:** La función listar_pacientes toma el control de la solicitud.

→ **Model:** Se invoca el método Paciente.objects.all() del modelo.

→ **Django ORM:** Traduce este comando a una consulta SELECT de SQL.

→ **SQLite:** Busca los registros en el archivo y devuelve los datos.

→ **Redirect / Render:** La vista usa la función render para mezclar los datos obtenidos con el HTML.

→ **Template:** El archivo listar.html recibe el contexto e itera sobre los pacientes para construir la tabla.

→ **Navegador:** Recibe el HTML procesado y dibuja la interfaz final para el usuario.

Flujo UPDATE (Editar Paciente):



→ **Navegador:** El usuario envía los datos corregidos mediante un POST desde el formulario prellenado.

→ **URL:** La ruta paciente/editar/<id>/ captura la petición extrayendo el ID del paciente.

→ **View:** La función editar_paciente asocia los nuevos datos a la instancia existente.

→ **Model:** Asegura que los tipos de datos modificados coincidan con los atributos.

→ **Django ORM: A**l llamar a save(), reconoce que el objeto tiene un ID existente y prepara la actualización.

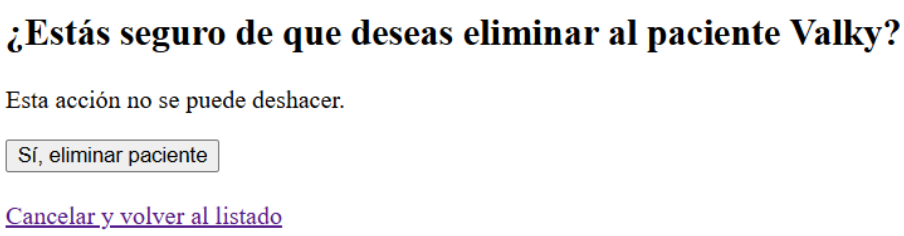
→ **SQLite:** Ejecuta el UPDATE sobrescribiendo únicamente los valores necesarios en esa fila.

→ **Redirect / Render:** La vista ejecuta un redirect hacia el listado general.

→ **Template:** Se recarga el HTML del listado.

→ **Navegador:** Muestra la tabla donde se aprecia el dato modificado correctamente.

Flujo DELETE (Eliminar Paciente):



→ **Navegador:** El usuario presiona el botón "Sí, eliminar", confirmando la acción con un POST.

→ **URL:** La ruta paciente/eliminar/<id>/ procesa el identificador.

→ **View:** La función eliminar_paciente recupera el objeto y llama a su método delete().

→ **Model:** Identifica el registro exacto dependiente de la clase Paciente.

→ **Django ORM:** Traduce la acción a una orden de borrado de base de datos.

→ **SQLite:** Ejecuta el comando DELETE borrando la fila de manera permanente.

→ **Redirect / Render:** Responde al cliente con un redirect al menú principal.

→ **Template:** El listado general se regenera sin el objeto eliminado.

→ **Navegador:** Renderiza la tabla donde se comprueba que el registro ya no existe.

RESULTADO FINAL:

Sistema de Gestión Clínica Veterinaria

+ Registrar Propietario | + Registrar Paciente

Pacientes Registrados (all)

Nombre	Especie	Raza	Propietario Responsable	Acciones
Valky	Perro	Bulldog Frances	Jesus Jose Castillo Sumire	Editar | Eliminar

Propietarios (order_by apellidos)

Apellidos	Nombres	DNI	Teléfono
Castillo Sumire	Jesus Jose	71541966	920434999

Medicamentos Disponibles (filter stock > 0)

Nombre	Stock	Precio
Amoxicilina	15	S/ 25.50
Antipulgas Pipeta	8	S/ 45.00

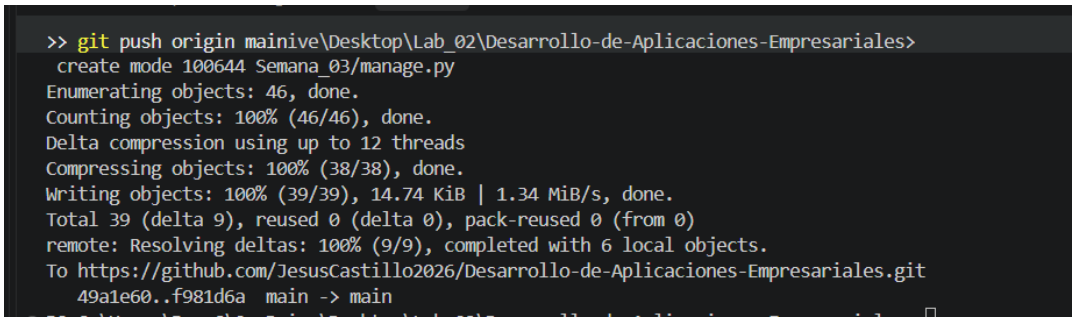
Servicios Clínicos (order_by precio)

Servicio	Duración (min)	Precio
Desparasitación Interna	15	S/ 25.00
Vacunación Antirrábica	20	S/ 35.00
Consulta General	30	S/ 50.00
Baño Medicado y Corte	60	S/ 65.00

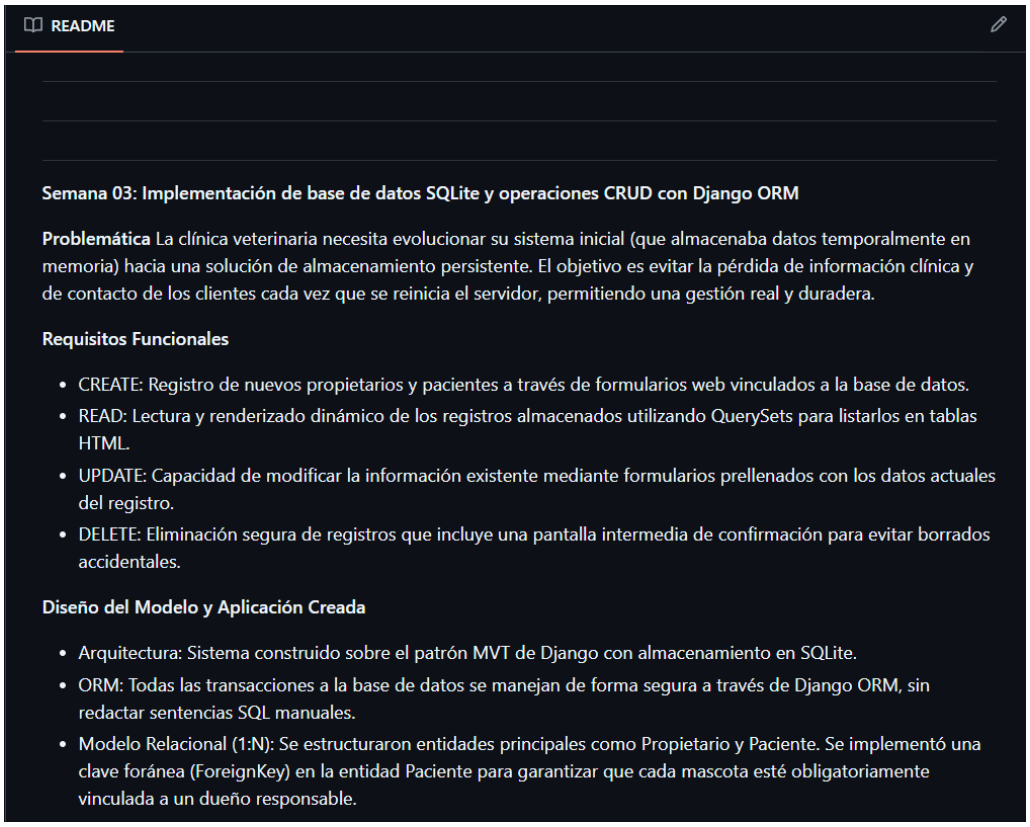
Ejercicio 10 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo su problemática, sus requisitos y la App creada. Hagan commit y push al repositorio del equipo.









Entregables

- Documento de requisitos: problemática elegida (Ejercicio 1) y lista de requisitos funcionales (Ejercicio 2).
- Diseño del modelo de datos: entidad principal, campos, tipos y justificación (Ejercicio 3).
- Código fuente completo de la nueva App: models.py (con los datos estáticos), views.py, urls.py, forms.py y templates (listado y formulario).
- Capturas de pantalla del flujo funcionando: listado de registros, formulario de creación, y el nuevo registro reflejado en el listado.
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo, según el formato de evidencia indicado en las instrucciones del laboratorio (nombre, título, captura, código, explicación y casos de prueba).

PLUS: MEJORA DE HTML

PROMPT UTILIZADO: USO DE GEMINI PRO

hola, ya tengo funcionando el crud de la semana 03 con los modelos de propietario, paciente, medicamento y servicio, pero la vista actual de listar.html y los formularios se ven muy planos y basicos (solo tablas con bordes simples).

ayudame a mejorar el diseño de los templates usando bootstrap o css moderno para que la interfaz de gestion clinica se vea profesional, limpia y ordenada.

necesito que mantengas la misma logica de django (los for, las variables de contexto y los enlaces a crear/editar/eliminar), pero que agregues:

- un diseño tipo dashboard con navbar superior

- tablas con mejor estilo, bordes redondeados y badges para los estados o stock

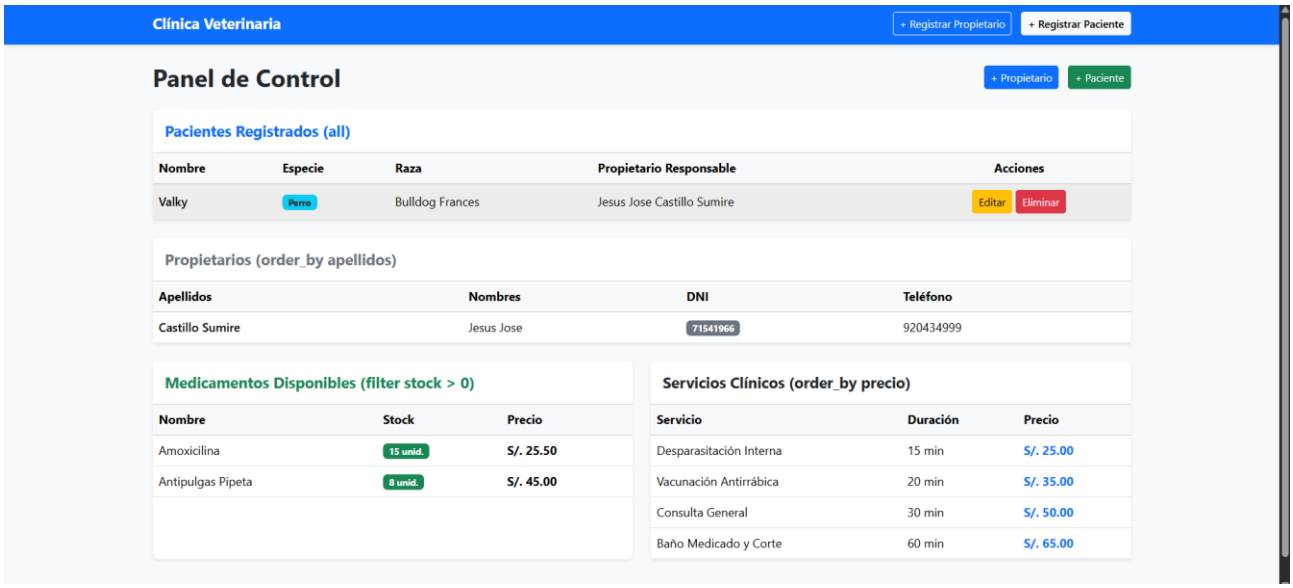
- botones modernos para las acciones (editar, eliminar, registrar)

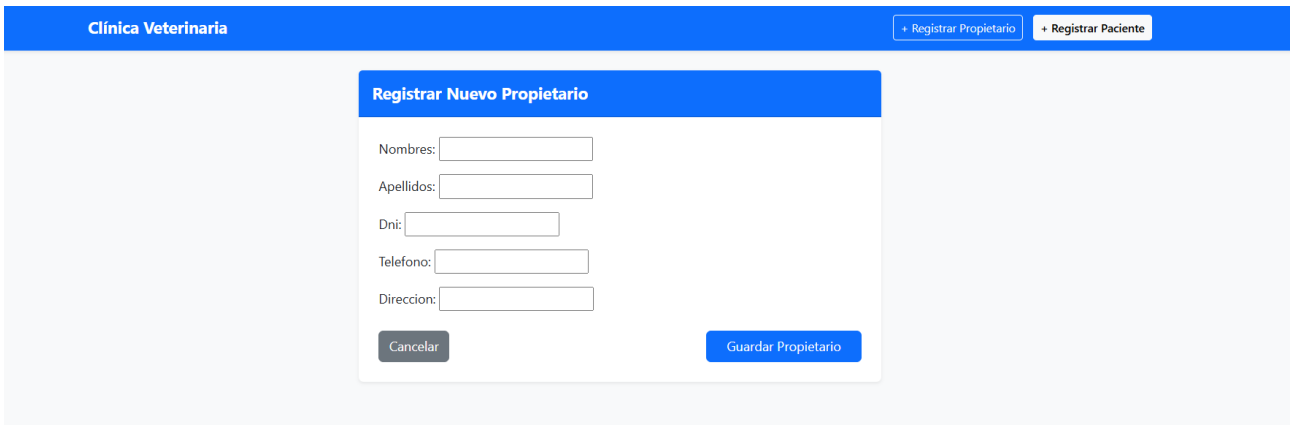
- formularios centrados tipo tarjetas (cards)

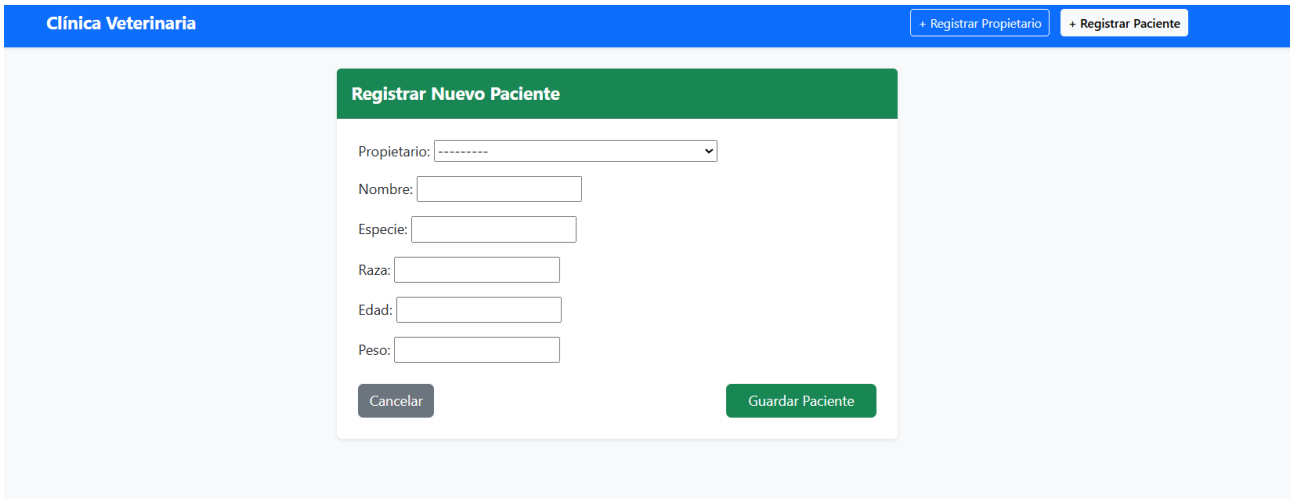
- que el codigo sea basico, facil de entender y sin explicaciones innecesarias

pásame el codigo completo de listar.html y de los formularios para reemplazarlo directamente en mi proyecto.

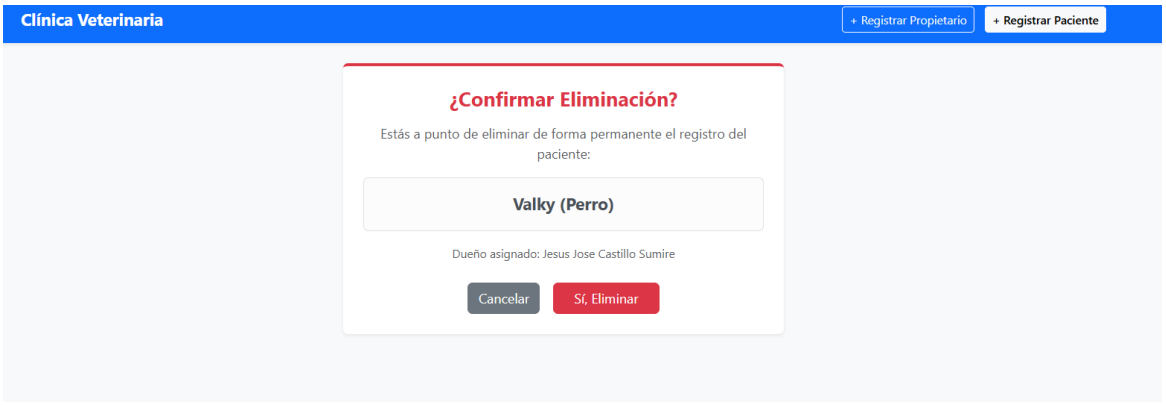
RESULTADO FINAL:











Conclusiones:

- Se implementó con éxito la arquitectura Modelo-Vista-Template (MVT) de Django en el proyecto Semana_03, logrando una gestión completa de las entidades de la clínica veterinaria (Propietario, Paciente, Medicamento y Servicio). Se validó la persistencia de datos relacionales mediante SQLite y el uso de consultas ORM como all(), order_by() y filtros condicionales (stock__gt=0), asegurando que las operaciones de creación, lectura, actualización y eliminación (CRUD) funcionen de forma íntegra.

- Se logró una notable modernización de la experiencia de usuario mediante la integración de Bootstrap y la creación de una plantilla base (base.html). Esto permitió centralizar elementos globales como la barra de navegación y los estilos responsivos, optimizando la reutilización de código y transformando formularios y tablas planas en una interfaz tipo dashboard ordenada, clara y con retroalimentación visual mediante tarjetas y botones de acción.

- Como observación técnica fundamental, se evidenció la importancia de mantener una rigurosa concordancia entre las rutas definidas en urls.py, las funciones controladoras en views.py y las referencias invocadas en los templates a través de la etiqueta {% url %}. Discrepancias mínimas en los nombres de las rutas asignadas pueden derivar en excepciones como NoReverseMatch, redirecciones a vistas no deseadas o confusiones entre las acciones de edición y eliminación, por lo que una convención de nombres clara y un mapeo cuidadoso de parámetros resultan críticos para la estabilidad del backend.

GITHUB:

https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales.git